

Final Exam — Project Exercises

v1.0

Course: High Performance Computing 1 / Introduction to Parallelism @ University of Trieste, 2025-2026

Teachers:

Luca Tornatore, INAF - luca.tornatore@inaf.it

Giuliano Taffoni, INAF - giuliano.taffoni@inaf.it

Stefano Cozzini, Area di Ricerca - stefano.cozzini@areasciencepark.it

Format: choose **one** of the three exercises below, implements a parallel HPC code using MPI + OpenMP, performs strong- and weak-scaling experiments, and discusses bottlenecks, optimisation choices, and pitfalls during the oral exam.

Each exercise has three layers:

- a **baseline** to fast your work and make you focusing on HPC and parallelism
- an **optimisation envelope** — non-obvious performance issues thst you are expected to find, measure, and discuss;
- an **open-ended part** for the stronger students (and the basis of the harder oral-exam questions).

IMPORTANT NOTE:

Given the timing and the lack of computational resources, there is a simplified track (the track A of exercise 3) that should be reasonably fast. However, the trade-off – to be fair with the students that choose the more difficult tracks – is that out of the assignment you can get at maximum mark of 25/30. Of course, with more extensive discussion and theoretical questions, you can scale up to full mark.

This fast track may be tempting especially for those who aim at the session of June 22nd.

Final Exam — Project Exercises

Mandatory deliverables (all three exercises)

Exercise 1 — Direct N-body gravitational simulation

Problem

Algorithm

Required parallel structure

Optimisation hints

Scaling

Suggested oral discussion points

Container layer — N-body on LEONARDO via Singularity

Background

Dockerfile

Singularity definition file

Running on LEONARDO with Singularity and host MPI

Scalability experiments — container vs. native

Suggested oral discussion points (container layer)

Exercise 2 — Parallel sample sort

Problem

Algorithm — sample sort

- Required parallel structure
- Optimisation hints
- Scaling
- Suggested oral discussion points
- Container layer — sample sort on LEONARDO via Singularity
 - Background
 - Dockerfile
 - Singularity definition file
 - Running on LEONARDO
 - Scalability experiments — container vs. native
 - Suggested oral discussion points (container layer)

Exercise 3 — Mandelbrot rendering with dynamic load balancing

- Problem
- Command-line interface
- Two tracks — with different scoring caps
- Algorithm
 - Track A — static stripes with dynamic dispatch
 - Track B — Mariani–Silver with adaptive subdivision
- Required parallel structure
- Result recollection
 - Track A — direct write via MPI-IO
 - Track B — two recollection variants
- Optimisation hints
- Scaling
- Suggested oral discussion points
- Container layer — sample sort on LEONARDO via Singularity

Appendix to Exercise 3 — Further possible optimisations

- Algorithmic acceleration
- Compute kernel
- Distribution and I/O
- Hardware

Mandatory deliverables (all three exercises)

Regardless of which exercise is chosen, the project report must include the following:

- 1. Hardware identification:** CPU model, number of sockets, cores per socket, SMT on/off, NUMA layout (`numactl -H` output or equivalent), memory configuration,...
- 2. Software stack:** compiler and version, MPI implementation and version, OpenMP runtime, BLAS or any library used.
- 3. Compiler flags,** exactly as used. If `-O3` and `-march=native` are used, this must be stated; if profile-guided or feedback optimisation is used, the procedure must be described.
- 4. Run configuration:** number of processes, threads per process, `OMP_PLACES`, `OMP_PROC_BIND`, `MPI_BIND` or equivalent
- 5. Verification:** a quantitative correctness check that does not rely on wall-clock measurement (energy conservation, L2-norm of the residual, checksum of the sorted array, etc.). *Some hints are found in the baselines.*

6. **Statistical treatment:** each measurement is the median (or, better, the trimmed mean and standard deviation) of at least five repetitions. Outliers and warm-up runs should be reported and discussed.
7. **Strong-scaling and weak-scaling** plots, each with parallel efficiency alongside speedup, and with a mention of where Amdahl/Gustafson is stepping in.
8. **Profiling evidence** for the bottleneck claim: we did not cover tools for that in the course, but you can instrument your code. Give "memory bandwidth"-style measurements of performance.
9. **Container overhead table:** a side-by-side comparison of wall-clock time (median $\pm \sigma$ over five runs) for the native binary versus the Singularity-wrapped binary at identical process counts, with the relative overhead expressed as a percentage. The table must cover at least three different process/node configurations. Overhead that is not zero must be explained — not dismissed.

Exercise 1 — Direct N-body gravitational simulation

Problem

Simulate the gravitational evolution of N point masses in three-dimensional space, with a softened potential and open boundary conditions:

$$\mathbf{F}_i = G m_i \sum_{j \neq i} m_j \frac{\mathbf{r}_{ij}}{(|\mathbf{r}_{ij}|^2 + \varepsilon^2)^{3/2}},$$

where ε is the softening length (used to keep the force finite for close encounters; it is *not* a numerical trick to be hidden, it changes the physics).

Integration uses a second-order leapfrog (Kick-Drift-Kick), which is symplectic and gives clean energy diagnostics. The student must verify that the total energy $E = T + U$ is conserved to a fixed relative tolerance — typically 10^{-3} over the whole run for a Plummer sphere of $N = 10^4$ particles with appropriate softening.

Initial conditions: a Plummer sphere, or a uniform random ball with a Maxwellian velocity distribution. A small generator can be provided.

Suggested problem sizes: $N \in [10^4, 10^6]$ for strong scaling; per-process particle count $N/P \in [10^4, 10^5]$ for weak scaling.

Algorithm

Direct $\mathcal{O}(N^2)$ summation. No tree code, no fast-multipole, no PM.

The reason is *not* that this is the best algorithm — clearly it is not — but that it exposes a clean compute-bound problem on which vectorisation, FMA throughput, and AoS-vs-SoA layout choices are visible without being masked by the irregular structure of a tree.

Required parallel structure

- **MPI:** a *ring-shift* communication pattern. Each process owns N/P particles permanently (its "home" chunk) and rotates a "buffer" chunk through all $P - 1$ other processes; on each ring step, forces between the home chunk and the buffer chunk are accumulated.
- **OpenMP:** parallelisation of the force loop, with attention to the accumulator pattern (avoiding atomics in the inner loop).
- **Hybrid:** one MPI rank per NUMA node is the obvious starting configuration; students should also try one rank per socket and one rank per core and *measure* the difference.

Optimisation hints

The student is expected to confront, measure, and explain at least the following.

- **Newton's third law.** Using $\mathbf{F}_{ij} = -\mathbf{F}_{ji}$ halves the arithmetic but creates a write conflict on the force accumulator of particle j when the inner loop is parallelised. This is a real trade-off, not a free win: on what conditions does the saving in flops outweigh the overhead of the conflict resolution? When does it not?
- **AoS vs SoA.** A `particle_t {x,y,z,vx,vy,vz,m}` struct is the natural object but is a vectorisation killer. Convert to separate arrays for x, y, z , measure with PAPI counters or `perf`, and report.
- **Reciprocal square root.** The inverse $1/(r^2 + \varepsilon^2)^{3/2}$ has a Newton-iteration-refined `rsqrt` implementation that is roughly 2–3× faster than the libm path on most x86 cores. Discuss the accuracy implications.
- **FMA throughput and the critical path.** The accumulation $\mathbf{F}_i += m_j \mathbf{r}_{ij}(\dots)^{-3/2}$ chains through the same accumulator. Show why multiple partial accumulators improve throughput, and at what point the gain saturates.
- **Communication-computation overlap.** The next ring chunk can be prefetched with `MPI_Isend` / `MPI_Irecv` while the current chunk is being processed. Measure the overlap actually achieved (it is rarely as good as the napkin estimate predicts; explain why).
- **Vectorisation report [optional, not covered in the course].** Run with `-fopt-info-vec` or equivalent, find which loops the compiler vectorised, and confirm with hardware counters (`fp_arith_inst_retired.512b_packed_double` etc.). The gap between *vectorised* and *effectively vectorised* is often surprising.

Scaling

- **Strong scaling:** fix N (e.g. 10^5) and vary P . Discuss the floor where N/P becomes too small for SIMD utilisation, and the floor where the ring latency dominates.
- **Weak scaling:** fix N/P (e.g. 10^4 per process) and vary P . Total computation grows as $w \cdot N = w^2 P$; total communication per step grows linearly in P . So the compute-to-communication ratio is constant — *in theory*. Discuss why measurements depart from this and what wins eventually (network injection rate? OS jitter? load imbalance from non-uniform positions?).

Suggested oral discussion points

- Why does increasing ε change *both* the physics and the computational cost? What is the relationship to the typical interparticle distance?

- On a single socket, what determines the peak FLOP/s of your kernel?
- If N is increased by 10, by what factor should you expect the time per step to grow? Did it? Why or why not?
- If you used `rsqrt`, how do you know your energy-conservation diagnostic is actually testing the integrator and not being saturated by the approximation error of `rsqrt`?

Container layer — N-body on LEONARDO via Singularity

Background

N-body is a *compute-bound* kernel: nearly all wall-clock time is arithmetic, and MPI communication is a small ring-shift of N/P particles per step. This makes it an almost ideal test case for assessing the pure launch and library overhead of containerisation with minimal confounding from network effects. The student should exploit this: if container overhead is non-negligible here, it cannot be dismissed as a network artefact.

Dockerfile

The student must write a `Dockerfile` that produces a self-contained image with all build-time dependencies. On the Lecture notes students can find dockerfile examples suitable for this exercise

Key decisions the student must justify in the report:

- Why `ubuntu:22.04` and not a vendor HPC image (e.g. `nvcr.io/hpc/...`)?
- Why is OpenMPI installed *inside* the container if LEONARDO will provide its own MPI at runtime? (Hint: the container MPI is used only for the build; at runtime Singularity binds the host MPI over it — see §4.3.)
- Why `-march=x86-64-v3` and not `-march=native`? What performance is left on the table, and how would you measure it?

Singularity definition file

Convert the Docker image to an Singularity `.sif` via the definition file below, or pull the Docker image directly:

```
# Option A: build from the definition file (preferred – gives full control)
singularity build nbody.sif nbody.def

# Option B: convert from Docker Hub or local daemon
singularity build nbody.sif docker-daemon://nbody:latest
```

Reference definition file (`nbody.def`) can be found in the lecture notes.

Running on LEONARDO with Singularity and host MPI

LEONARDO uses OpenMPI or Intel MPI depending on the module environment. The critical point is that **Singularity must bind the host MPI** to the container, otherwise multi-node parallelism either fails silently or falls back to shared memory. The student must understand and document this mechanism.

The Reference Slurm batch script can be found in the lecture notes adn adapted for this exercise.

The student must verify with `ldd` inside and outside the container that the MPI library being loaded at runtime is the host library, not the container's built-in one. A mismatch here is a common source of silent performance degradation or hangs.

Scalability experiments — container vs. native

The N-body kernel is compute-bound; the main container overhead to expect is:

- **Launch overhead:** Singularity container startup (typically 0.5-2 s). This is a one-time fixed cost; it becomes negligible for long runs but dominates short benchmarks. Measure it explicitly with `time singularity exec ... true`.
- **Library binding overhead:** if MPI binding is imperfect, you may see slightly higher latency in the ring-shift communication. Measure with a micro-benchmark (e.g. `osu_latency` and `osu_bw` from the OSU suite, both native and containerised).
- **Compilation target mismatch:** if the container binary was built with `-march=x86-64-v3` but the host supports AVX-512 (`-march=sapphirerapids`), there will be a measurable throughput gap in the SIMD-heavy inner loop.

Required experiments:

Experiment	Fixed parameters	What varies
Strong scaling — native	$N = 10^5$, 100 steps	$P \in \{1, 2, 4, 8, 16, 32\}$
Strong scaling — container	same	same
Weak scaling — native	$N/P = 10^4$, 100 steps	$P \in \{1, 2, 4, 8, 16\}$
Weak scaling — container	same	same
Launch overhead	1 process	10 repeated launches, <code>time singularity exec</code>
MPI micro-benchmark	OSU latency + bandwidth	native vs. container, 2 processes

For each experiment, report median $\pm \sigma$ over five runs. Plot native and container results on the same axes with distinct markers; annotate the percentage overhead at each point.

Expected finding: for this exercise the arithmetic intensity is high enough that container overhead should be below 2–5% for runs longer than ~30 s. If the student observes higher overhead, they must diagnose the cause.

Suggested oral discussion points (container layer)

- You used `-march=x86-64-v3` in the container. LEONARDO's nodes support AVX-512. Estimate the theoretical throughput penalty. How much did you actually measure? Why might the gap be smaller or larger than the theoretical estimate?
- Your container includes an OpenMPI installation, but at runtime the host MPI is injected. What happens if the container MPI version and the host MPI version are incompatible? How would you detect this without access to the source?
- If you ran the same `.sif` image on a different cluster with InfiniBand instead of LEONARDO's Cornelis OmniPath, what would you need to change? What would work unchanged?
- Is the N-body ring-shift pattern safe to containerise without any modification, or are there MPI features (e.g. shared memory segments, XPMEM) that may be unavailable inside the container?

Exercise 2 — Parallel sample sort

Problem

Sort a globally distributed array of N keys (32-bit or 64-bit integers, or double-precision floats — choose and justify), where each of the P MPI processes initially holds an unsorted chunk of approximately N/P elements. At the end, process r must hold a sorted subarray such that all elements on process r are less than or equal to all elements on process $r + 1$.

Verification: every output element must appear exactly once, and the global sequence must be non-decreasing. The student writes a parallel verifier.

Suggested sizes: N from 10^7 to 10^9 aggregate; per-process input size N/P in the range that fits comfortably in DRAM (not just in cache — otherwise you are not really doing a sort, you are doing a benchmark of a small problem).

Algorithm — sample sort

1. Each process sorts its local chunk in place.
2. Each process selects $P - 1$ regularly spaced samples from its sorted chunk (regular sampling, not random — this gives much better load balance).
3. All samples are gathered (`MPI_Allgather`), sorted, and from them $P - 1$ global pivots are selected.
4. Each process partitions its local data according to the pivots into P buckets.
5. Each process exchanges buckets with every other process (`MPI_Alltoall` for sizes, then `MPI_Alltoallv` for data).
6. Each process performs a k -way merge of the P incoming sorted streams (they are already sorted because the local chunks were sorted in step 1 and the buckets preserve ordering inside).

Required parallel structure

- **OpenMP** for the local sort, and for the k -way merge where applicable.
- **MPI** for the global communication: `MPI_Allgather` of the samples, `MPI_Alltoall` of the bucket sizes, `MPI_Alltoallv` of the bucket data.
- **Robust to imbalance.** If the input is drawn from a non-uniform distribution (the student should test with at least one skewed input: a Zipf, or a normal with heavy tails), the regular-sampling step still gives quasi-balanced output, but the imbalance does not vanish. Measure max / min / mean of the final per-process sizes and discuss.

Optimisation hints

- **Choice of local sort.** A radix sort beats a quicksort on uniform 32-bit keys by a factor that depends on the number of passes and the cache behaviour; on doubles or 64-bit keys, this is no longer obvious. The student picks one, justifies it, and ideally tries two.
- **Cache behaviour of the local sort.** Branch-rich sorts (quicksort) and branch-poor sorts (radix) have very different performance signatures.
[optional, not covered in course] Measure with `perf stat -e branch-misses,cache-misses` and explain.
- **All-to-all is the bottleneck.** Show the bandwidth achieved versus the injection rate of the network. For a node-internal benchmark, compare intra-node `MPI_Alltoallv` against an OpenMP "shared-memory all-to-all".
- **Memory allocation.** The size of each incoming bucket is not known a priori. The student must exchange sizes first with `MPI_Alltoall`, then allocate, then `MPI_Alltoallv`. Discuss the cost of this two-phase pattern and what could be done if memory pressure is a problem.
- **k -way merge.** The naïve approach merges two sorted runs at a time, $O(P \log P)$ passes through the data. A tournament tree / heap-based k -way merge does it in a single pass but with extra branch overhead. Measure.
- **Imbalance vs replication.** With regular sampling on a uniform input, the worst-case bucket size is bounded by roughly $2N/P$ (classical result).
Verify the bound empirically and discuss when it can be violated.

Scaling

- **Strong scaling:** fix N , vary P . Sample sort has $\mathcal{O}(N \log N/P)$ local work and $\mathcal{O}(N)$ aggregate communication; communication will dominate eventually. Find the cross-over for your machine.
- **Weak scaling:** fix N/P , vary P . Per-process work stays constant in the local sort phase; per-process communicated volume grows as $N(P-1)/P \rightarrow N$. So weak scaling is inherently bad at high P — show this and discuss whether a different sort (e.g. histogram sort or hyperquicksort) would do better and why.

Suggested oral discussion points

- If your input is *already sorted*, what does sample sort do, and is that optimal?
- The regular-sampling step picks P^2 samples globally. Why not pick P or

P^3 ? What is the role of the *oversampling factor*?

- Why is `MPI_Alltoallv` typically much slower per byte than `MPI_Alltoall` for the same total data volume?
- On a fat-tree network with L levels, what is the lower bound on communication time for `MPI_Alltoall` of total volume V per process? How close did you get?
- What happens to your sort if you replace `MPI_Alltoallv` with $\mathcal{O}(P^2)$ point-to-point messages? Predict and measure.

Container layer — sample sort on LEONARDO via Singularity

Background

Sample sort is *communication-dominated* at scale: the `MPI_Alltoallv` call exchanges $\mathcal{O}(N)$ data with all-to-all traffic, which saturates network injection bandwidth long before the compute bottleneck is reached. This makes it the most demanding of the three exercises for the container network path. A container that introduces even a small per-message overhead in `MPI_Alltoallv` will show it clearly in the scaling curves.

Dockerfile

Reference dockerfile can be found in the Lecture notes.

Notable addition: `libnuma-dev` should be included so that the student can call `numa_alloc_onnode()` explicitly if testing fine-grained NUMA placement inside the container. The student should document whether NUMA topology is visible inside the Singularity namespace (it usually is, via `/sys/devices/system/node/`).

Singularity definition file

Convert the Docker image to an Singularity `.sif` via the definition file below, or pull the Docker image directly:

```
# Option A: build from the definition file (preferred — gives full control)
singularity build heat.sif heat.def

# Option B: convert from Docker Hub or local daemon
singularity build heat.sif docker-daemon://heat:1.0
```

Reference definition file (`heat.def`) can be found in the lecture notes.

Running on LEONARDO

The Reference Slurm batch script can be found in the lecture notes and adapted for this exercise.

Scalability experiments — container vs. native

Sample sort's `MPI_Alltoallv` is acutely sensitive to network library injection quality. The student must probe multiple aspects:

- **All-to-all bandwidth:** use `osu_alltoallv` from the OSU suite, both natively and inside the container, for message sizes from 1 KiB to 64 MiB. A correctly injected MPI library should give identical results; any gap indicates incomplete binding.
- **Collective algorithm selection:** OpenMPI selects different collective algorithms depending on process count, message size, and network topology. Inside a container, the topology detection may differ (e.g., if `hwloc` or `fi_info` produces different output). Document which algorithm OpenMPI selected in each case with `--mca coll_tuned_use_dynamic_rules 1 -v`.
- **Imbalanced input sensitivity:** run the skewed-distribution test both native and containerised. The load imbalance should be identical (it is a function of the data, not the execution environment), but wall-clock time may differ if collective algorithms react differently.

Required experiments:

Experiment	Fixed parameters	What varies
Strong scaling — native	$N = 10^9$, uniform	$P \in \{4, 8, 16, 32, 64\}$
Strong scaling — container	same	same
Weak scaling — native	$N/P = 2 \times 10^7$, uniform	$P \in \{4, 8, 16, 32\}$
Weak scaling — container	same	same
Skewed input — native	$N = 10^8$, Zipf $\alpha = 1.1$	$P \in \{8, 16, 32\}$
Skewed input — container	same	same
OSU <code>osu_alltoallv</code>	2–64 ranks, intra+inter node	message 1 KiB – 64 MiB

Expected finding: this exercise is the most likely of the three to show measurable container overhead at large P , specifically in the `MPI_Alltoallv` phase. The student should isolate the communication time (e.g. by profiling with `mpiP`) and compare it to the local sort time, both native and containerised.

Suggested oral discussion points (container layer)

- Your container MPI and the host MPI may use different collective algorithms for `MPI_Alltoallv`. How would you detect this? Would it matter for correctness? Would it matter for performance?
- The OSU `osu_alltoallv` benchmark showed X% lower bandwidth inside the container compared with native. Which layer is responsible: the container

filesystem namespace, the MPI library binding, or the network driver injection? How would you systematically narrow it down?

- If LEONARDO migrated from OmniPath to Infiniband, what would need to change in your container image to get native network performance? What would not change?
- Sample sort's weak scaling is inherently poor because communicated volume per process grows with P . If you ran the same `.sif` on a cluster with twice the network bandwidth, by how much would the weak-scaling efficiency improve at $P = 64$?

Exercise 3 — Mandelbrot rendering with dynamic load balancing

Problem

Render the Mandelbrot set $\mathcal{M}$ on a rectangular region of the complex plane, $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$, discretised on an $N_x \times N_y$ pixel grid. For each pixel centre $c \in \mathbb{C}$, iterate

$z_{k+1} = z_k^2 + c$ from $z_0 = 0$ until $|z_k| > 2$ or $k = K_{\max}$.

The image's colour at the pixel is a function of the escape iteration count (typically $K_{\max}$ for points presumed in $\mathcal{M}$).

The naive renderer computes every pixel independently. The problem is embarrassingly parallel pixel-by-pixel, but has a brutal load imbalance: pixels deep inside $\mathcal{M}$ cost $K_{\max}$ iterations each, pixels far outside cost a handful. Any static decomposition gives terrible parallel efficiency, because the regions of high cost are highly non-uniform and do not average out across a small number of processes.

Suggested image sizes: $N_x = N_y$ between 1024 and 16384.

Suggested $K_{\max}$ between 10^3 and 10^5 . The view should be chosen to include a non-trivial fraction of $\mathcal{M}$ (the standard $[-2.5, 1.5] \times [-2, 2]$ box, or a "deep zoom" near the boundary where the filament structure is visible).

Command-line interface

Both Tracks must accept the following command-line arguments, with the defaults shown in parentheses. Flag-based parsing (e.g. `--ppu 256`) is recommended for usability; positional parsing is acceptable if documented in the report.

The interface is common to both Tracks so that students can swap implementations without changing the launcher.

- `--xmin, --ymax`: real and imaginary coordinates of the *upper-left* corner of the view (defaults -2.5 and 2.0).
- `--xmax, --ymin`: real and imaginary coordinates of the *bottom-right* corner (defaults 1.5 and -2.0).
- `--ppu`: pixels per unit length on the real axis, applied identically to both axes so that pixels are square in the complex plane (default 256).

This fixes the resolution: $N_x = \text{ppu} \cdot (x_{\max} - x_{\min})$ and $N_y = \text{ppu} \cdot (y_{\max} - y_{\min})$.

- `--dx-factor, --dy-factor`: integer factors f_x, f_y determining the initial tile size (defaults

$$f_x = f_y = 8).$$

Track B uses both: $D_x = N_x/f_x$, $D_y = N_y/f_y$.

Track A uses only f_y — stripes span the full image width — and f_x is ignored.

- `--kmax`: maximum iteration count $K_{\max}$ (default 1024).

With the defaults, the standard view is rendered at 1024×1024 pixels, with $f_x = f_y = 8$ giving initial tiles of 128×128 (Track B) or stripes of 1024×128 (Track A), and $K_{\max} = 1024$ iterations per pixel.

Two tracks — with different scoring caps

The exercise admits two Tracks that differ fundamentally in *algorithmic* content. The student picks one and declares it at the beginning of the report.

- **Track A — static horizontal stripes (simple) : MAXIMUM mark for the assignment is 26.**

The image is divided into f_y static stripes of size $N_x \times D_y$ where $D_y = N_y/f_y$. The producer (rank 0) holds this fixed queue and hands stripes out to consumer ranks on request. Each consumer computes every pixel of its stripe by direct iteration — *no Mariani-Silver, no subdivision* — then writes the stripe directly to the output file at the known byte offset via `MPI_File_write_at`. When the queue is empty, the producer replies "done" to subsequent requests and exits when all consumers have acknowledged.

The point of Track A is the *parallelisation pattern*, not the algorithm: dynamic producer-consumer dispatch, OpenMP inside a stripe, MPI-IO for output. The load imbalance comes from the natural per-pixel cost variation across stripes (some stripes contain many pixels deep in $\mathcal{M}$, others mostly escape early), and is largely absorbed by the dynamic dispatch.

Maximum grade from the project: 25.

The remainder of the mark may come from oral-exam performance on theory questions.

- **Track B — Mariani-Silver with adaptive 2D subdivision.**

As described in the Algorithm section below: tiles are subdivided into four quadrants (NW/NE/SW/SE) when the boundary heuristic does not apply, with the consumer keeping NW and the producer queueing the other three.

Completed tiles are sparse 2D patches: here the recollection is non-trivial and is itself part of the exercise.

For this Track the *load imbalance between collectors* (message-count imbalance, since a stripe in a filament-rich region collects many small messages while a uniform stripe collects few large ones) is explicitly **out of scope**: a uniform stripe partition is acceptable.

The grade cap on Track A comes from the fact that the static-stripe variant is meaningfully easier to implement and discuss.

That is to ease those of you that want to take the exam earlier, at the cost of having a lower final mark.

Algorithm

Track A — static stripes with dynamic dispatch

The producer (rank 0) initialises a queue of f_y stripes $\{S_0, S_1, \dots, S_{f_y-1}\}$, where stripe S_i spans pixel rows $[i \cdot D_y, (i+1) \cdot D_y)$ and the full image width N_x .

Consumers request a stripe; the producer pops one off the queue and ships it; the consumer computes every pixel by direct iteration to $K_{\max}$ and writes the resulting $D_y \cdot N_x$ pixels to the output file via

`MPI_File_write_at` at offset $i \cdot D_y \cdot N_x \cdot s$ (with s the bytes-per-pixel size). When the producer's queue is empty, it replies "done" to subsequent requests and exits when every consumer has acknowledged.

No subdivision, no boundary heuristic, no central pixel sink. This is the simplest non-trivial producer-consumer pattern, and the point of Track A is to implement it cleanly and measure it carefully.

Track B — Mariani-Silver with adaptive subdivision

- **Mariani-Silver boundary tracing.**

Test only the pixels on the perimeter of a tile.

If they all stay bounded under iteration up to $K_{\max}$ (all classified as "in $\mathcal{M}$ "), then *the entire tile is in $\mathcal{M}$* : colour all interior pixels with the inside colour and skip them.

If they all stay unbound, give to the entire tile the average value along the perimeter (possible refinement: subdivide in 4 smaller tiles, calculate the perimeter of each and give to each of them the average of their own perimeter).

In the mixed case, the MPI rank rejects the tile, and signal that to the producer rank.

Note: *This is mathematically justified by the fact that $\mathcal{M}$ is simply connected (Douady-Hubbard, 1982): its complement in $\mathbb{C}$ is connected and unbounded, so a "bubble" of complement cannot be trapped inside a rectangle whose boundary is entirely in $\mathcal{M}$. The caveat is that we sample the perimeter at pixel resolution: a filament of complement thinner than a pixel can pierce the rectangle between two sampled boundary points, leaving the interior misclassified. The error shrinks with resolution and is invisible to the eye at typical viewing sizes, but it is real and must be measured.*

Note the asymmetry: the shortcut helps for tiles *inside* $\mathcal{M}$, where each pixel would otherwise cost $K_{\max}$ iterations; tiles *outside* $\mathcal{M}$ terminate fast pixel-by-pixel and the shortcut saves little.

The heuristic is itself essentially free if the tile is accepted — the perimeter pixels would be computed anyway in the brute-force pass — so its expected payoff is non-negative regardless of the view.

If the tile is to be rejected, the heuristic is costly: can you imagine how to minimize the cost?

- **Producer-consumer with adaptive subdivision.**

The plane is tiled initially into a $f_x \times f_y$ grid of tiles, each of size $D_x \times D_y$.

Rank 0 is the producer: it holds the work queue and dispatches tiles on consumer request.

Each consumer (rank ≥ 1), upon receiving a tile:

1. Computes the iteration count of every pixel on the tile's perimeter.
2. If all perimeter pixels are in $\mathcal{M}$, fills the interior with the inside colour and reports the tile complete.
3. Otherwise:
 - if the tile is smaller than a threshold τ (linear pixel size), computes every interior pixel directly and reports the tile complete;
 - otherwise, sends a *subdivision request* to the producer. The producer queues the NW, NE, SW, SE quadrants for later dispatch.
Possible optimization:
the consumer proceeds with the - say - NW quadrants without waiting for the Producer to assign something, and the producer register that accordingly. The local recursion on the NW quadrant is better implemented as an explicit stack-based loop, not function recursion: deep zooms can drive the recursion depth into the tens, which is fine for a stack but ugly with C function calls.

Required parallel structure

- **Producer (rank 0).** Maintains the work queue. In Track A the queue is the initial set of f_y stripes and only shrinks. In Track B the queue also contains subdivided quadrants pushed back from consumers, and grows during execution; the producer additionally tracks an in-flight count (tiles dispatched but not yet reported complete) and receives the completed tiles' pixel data along the recollection path (see *Result recollection*).
- **Consumers (rank ≥ 1).** Request work, process it, report result, request next. In Track A "process" means iterating every pixel of the stripe. In Track B "process" means running the Mariani–Silver test and either computing the interior, computing brute-force, or recursing internally on the NW quadrant (with a subdivision request to the producer for NE/SW/SE).
- **OpenMP inside each stripe/tile.** The per-pixel computation is embarrassingly parallel but with wildly non-uniform per-pixel cost (some pixels reach $K_{\max}$, others escape in a few iterations). Static scheduling is pathological; `schedule(dynamic, chunk)` or `schedule(guided)` is needed. Chunk size must be tuned.
- **Termination detection.**
 - *TrackA:* the queue starts with f_y stripes and only shrinks, so "queue empty" is a sufficient termination condition. The producer replies "done" to requests arriving after the queue empties, and exits once every consumer has acknowledged.
 - *TrackB:* the queue can grow (subdivisions add tiles) while consumers are working, so termination requires both `queue_empty` and `in_flight_count == 0`. Declaring termination on "queue empty" alone is the canonical bug, because an in-flight tile may still generate subdivisions and re-fill the queue.

Result recollection

The mechanism by which completed-tile pixel data reaches the final image depends on the chosen Track.

Track A — direct write via MPI-IO

Every stripe spans a contiguous range of full image rows. Each consumer, upon completing stripe S_i (rows $i \cdot D_y$ to $(i + 1) \cdot D_y$), writes the corresponding $D_y \cdot N_x$ pixels directly to the output file at byte offset $i \cdot D_y \cdot N_x \cdot s$ (where s is the bytes-per-pixel size) using `MPI_File_write_at`. No central sink, no collectors. The producer is involved only in stripe dispatch and termination; it does not see pixel data at all.

Note: two consumers never write overlapping rows by construction (each stripe occupies a unique row range), and `MPI_File_write_at` is safe for non-overlapping concurrent writes — that is exactly the use case (ease your life) for which it exists.

Track B — two recollection variants

Completed tiles are sparse 2D patches; recollection requires a design choice.

Pick one of the two variants below, implement it, and discuss the trade-off in the report.

- **Single-sink variant.** Each consumer attaches the completed tile's pixel buffer to its "tile complete" message; the producer files the pixels into a global image buffer that it owns. Simple, with the lowest

implementation cost. The drawback is that rank 0 is already serving dispatch requests *and* receiving subdivision signals, and now also absorbs the entire pixel-data stream. At large P this triple role makes rank 0 the bottleneck before the network or compute do.

- **Distributed-stripes variant.** Logically partition the image into S horizontal stripes, where $S \leq P - 1$ (rank 0 stays pure-producer; you might change that) and the stripe height is an integer multiple of the initial tile height D_y . Each stripe is owned by exactly one collector rank $(1, 2, \dots, S)$.

When a consumer completes a tile whose bounding box has bottom at y , it sends the pixel buffer directly to the owner of the stripe containing y , computed as $\lfloor y/(mD_y) \rfloor + 1$, where mD_y is the stripe height.

The " $m \cdot D_y$ " constraint is essential. Initial tiles are aligned to multiples of D_y in y , and every subdivided quadrant is a strict subset of its parent initial tile. Therefore every tile at every recursion depth lies entirely within one initial-tile row, and a stripe height that is a multiple of D_y guarantees that no tile ever straddles two collectors.

How managing the **consumer-collector multi-identity** is an open challenge. Of course you can wait the end if everything, or you can overlap communication and computation (there a pi-greek calculation example among the MPI example code that may help in clarifying).

For the final assembly to a single file you may use MPI-IO: each collector writes its stripe to the correct file offset, in parallel.

Optimisation hints

You are expected to confront, measure, and discuss the items below. Items tagged with a Track apply only to that Track; the rest apply to both.

- **Producer bottleneck.** With P consumers each completing a stripe (Track A) or tile (Track B) every t ms, the producer must serve a request roughly every t/P ms. At large P or small t (small stripes/tiles, low $K_{\max}$, fast pixels), the producer's message-handling latency may become the floor of the entire scaling curve.

Measure the producer's occupancy (fraction of wall time spent in MPI calls or in dispatch logic) as a function of P .

Can you implement a mitigation?

- **OpenMP scheduling inside a stripe/tile.** Compare `static`, `dynamic`, and `guided` schedules. With `static`, one thread typically ends up with all the "deep in $\mathcal{M}$ " pixels and the others sit idle. Quantify the per-thread time imbalance with each schedule.
- **Conjugate symmetry.** $\mathcal{M}$ is symmetric under $c \mapsto \bar{c}$. If the view is symmetric about the real axis, the upper half-plane suffices and the lower is reflected — a factor of two for essentially free. Mention when this is applicable (and when it is not, e.g. a deep zoom on a feature not on the real axis).
- **Tuning the dispatch granularity.**
 - *Track A:* f_y is the only knob. Too small values lead to coarse stripes, large imbalance, few units of work to dispatch. Too large values lead to many small stripes, producer overhead dominates. Sweep over a range and justify the chosen value.
 - *Track B:* f_x , f_y , and τ are the knobs. if τ is too small, there is recursion overhead and message volume dominates. At odds, if τ is too large, you miss shortcut opportunities, and there is longer brute-force computations of tiles that should have subdivided.

As for f_x, f_y : too small $\Rightarrow$ coarse initial tiles, large imbalance until subdivision kicks in. f_x, f_y too large $\Rightarrow$ huge initial queue, producer buried in dispatch.

- (Track B) **Termination protocol.** Sketch your protocol explicitly and convince yourself it is race-free. Test it on a deliberately small image where the recursion produces deep trees on one tile while other consumers are idle.
- (Track B) **Verification against brute force.** The brute-force renderer provided (every pixel computed individually, no Mariani–Silver) is the ground truth at the chosen $K_{\max}$. You may compare pixel by pixel and report the mismatch rate as a function of image resolution. The mismatch rate should fall as resolution increases, because thinner filaments become resolvable.

Scaling

- **Strong scaling.** Fix the image and view, vary P . For Track B, choose a view with non-trivial structure (some "inside" mass, visible filaments — the standard view or a moderately deep zoom) so the heuristic actually does work. Plot speedup and efficiency. The producer's bottleneck is expected to appear at high P in both Tracks: identify it, predict where it bites, and confirm.
- **Weak scaling.** Fix per-consumer pixel budget (e.g. by scaling $N_x = N_y \propto \sqrt{P-1}$, since one rank is the producer), vary P . Per-consumer compute is roughly constant; producer load grows with P . The knee of the weak-scaling plot reveals the producer's limit.
- **Load-balance metrics.** For each P , report per-rank wall time (max, min, mean, standard deviation), number of stripes/tiles processed per rank, and aggregate pixels per rank. Imbalance below $\sim 10\%$ at large P is the target.

Suggested oral discussion points

These are just to stimulate your thinking, we're not saying that we will for sure ask these questions

- (Track B) The Mariani–Silver heuristic is sound because $\mathcal{M}$ is simply connected. Where exactly does the argument break down at finite pixel resolution? What kind of features does it miss?
- If you doubled $K_{\max}$, what would happen to the running time of your implementation?
- (Track B) Sketch a worst-case input for your termination-detection protocol. Walk through why it does not deadlock or terminate prematurely.
- For your producer–consumer design, what is the smallest stripe/tile size at which the round-trip with the producer is worth incurring? Estimate from your machine's MPI latency, then check against your measurement.
- If rank 0 also computed when its queue was empty, what would go wrong? (Hint: it cannot serve a request while it is in a long-running pixel iteration.)
- (Track A) You used `MPI_File_write_at` for output. What guarantees that two consumers' writes do not interfere? Did you observe any I/O bandwidth saturation in your scaling plots? At what $N_x \times N_y$ would you expect the parallel file system to start pushing back?
- (Track A) If you doubled f_y at fixed P , how would the producer's occupancy change? At fixed f_y , how would it change if you doubled P ?
- (Track B) If you implemented both recollection variants (single-sink and distributed-stripes): at what P

does the distributed variant start to beat the single-sink on your machine, and why? If you only implemented one, try to figure the cross-over from first principles (network injection rate of rank 0, average tile byte size, average number of tiles per consumer per second).

- (Track B) In the distributed-stripes variant, a rank is both a consumer (computing tiles) and a collector (receiving tiles from others). How did you ensure that incoming messages do not block the senders while you are deep in a pixel iteration?

Container layer — sample sort on LEONARDO via Singularity

TBD - will appear shortly, will be very similar to that for NBODY

Appendix to Exercise 3 — Further possible optimisations

This is a non-exhaustive list of optimisations but are not required (and would be impractical to require) within the scope of this exercise.

They are listed so the most curious know the territory and can pick one for deeper individual study, a thesis topic, or simply for curiosity.

Algorithmic acceleration

- **Cardioid and period-2 bulb tests.** Closed-form expressions decide membership in the main cardioid (the large central body of $\mathcal{M}$) and in the period-2 bulb (the disk to its left) without any iteration. They classify a significant fraction of typical-view interior pixels in $O(1)$ instead of $K_{\max}$. Very cheap to add, very effective for views containing significant cardioid or bulb mass.
 - **Periodicity detection.** A point is in $\mathcal{M}$ iff its orbit is bounded; in practice, a bounded orbit becomes periodic. Detecting periodicity (by comparing z_k to a stored reference z_{k_0}) lets one terminate the iteration of an in- $\mathcal{M}$ pixel after just a few hundred iterations instead of $K_{\max}$. Cost: a small extra storage per pixel and a comparison every few iterations.
- **Distance-estimator method.** Iterating $z'_{k+1} = 2z_k z'_k + 1$ alongside the main orbit gives an estimate of the distance from c to $\partial\mathcal{M}$. Enables both early termination ("clearly far from boundary") and smooth boundary rendering. Standard references: Milnor, *Dynamics in One Complex Variable*; Hubbard's lectures.
- **Perturbation theory and series approximation for deep zooms.** Beyond the double-precision floor ($\sim 10^{-15}$), compute one high-precision reference orbit and approximate nearby orbits as low-precision deltas (Pauldelbrot, K. I. Martin). Enables deep zooms at affordable cost; intricate enough to be a small research project in its own right.
- **Symmetric Mariani-Silver.** The shortcut described in this exercise catches "all perimeter in $\mathcal{M}$ " but not "all perimeter out".

The latter is also mathematically sound (by connectedness of $\mathcal{M}$) but saves much less time, because outside pixels are cheap. Still, for boundary-poor views it would close the asymmetry.

Compute kernel

- **SIMD with mask registers (AVX-512, ARM SVE).** Mask-register predication lets escaped lanes be masked off without branching, simplifying the refill strategy at the assembly level. Roughly equivalent expressivity to the manual refill scheme but with less bookkeeping.
- **Mixed precision.** Use single precision for the initial perimeter test (sufficient for the binary in/out decision); switch to double only for pixels near the boundary or for tiles flagged for brute-force computation.
- **Conjugate symmetry across the real axis.** Halves the work when the view is symmetric about $\text{Im}(c) = 0$. Free except for bookkeeping; inapplicable to deep zooms away from the real axis.
- **NUMA-aware tile staging.** For very large tiles (larger than the per-thread L2), allocate the tile buffer on the local NUMA node of the thread that will compute it. Usually irrelevant at the tile sizes this exercise uses, but listed for completeness.

Distribution and I/O

- **Hierarchical work distribution.** Multiple producers arranged in a tree, each serving a subset of consumers. Mitigates the rank-0 bottleneck at very large P at the cost of cross-producer coordination.
- **Work stealing.** Each rank maintains a local work queue and steals from other ranks' queues when its own is empty. Eliminates the central producer entirely; classical Cilk-style design. Hard to combine cleanly with the subdivision-into-the-producer model, but possible.
- **Space-filling-curve dispatch.** Issue tiles in an order that preserves spatial locality (Z-order, Hilbert), so consumers tend to receive spatially contiguous tiles. Helps with cache reuse and reduces the variance in per-consumer compute time.
- **Adaptive stripe partitioning (Track B).** Instead of equal-height stripes, partition so that each collector receives roughly the same number of *messages* by predicting filament density from the first few rounds of subdivisions. Requires either an online estimator or a preliminary low-resolution pass to predict the distribution. This is the imbalance issue declared out of scope in Track B; for a thesis-level student it would be a natural extension.
- **Collective MPI-IO with hints.** Tuning `striping_factor`, `striping_unit`, and `cb_nodes` for the underlying ROMIO can change output bandwidth by a factor of several on parallel file systems.

Hardware

- **GPU offload (CUDA / HIP / SYCL).** Mandelbrot is the textbook embarrassingly parallel GPU kernel; a single modern GPU can outrun a small CPU cluster on this workload. Outside the scope of this course.
- **FPGA / specialised hardware.** Possible but exotic; mentioned for completeness.